



IMI GUARD

OPERATOR MANUAL

Version 3.0 — "Unauthorized-Use Investigation" release
Windows 10 / 11 · Windows Server 2016+
Portable, read-only endpoint security audit & investigation tool

IMI TECH LAB · CYBERSECURITY & INNOVATION

IMI Guard — Operator Manual

Version: 3.0 ("Unauthorized-Use Investigation" release) **Platform:** Windows 10 / 11, Windows Server (2016+) **Type:** Portable, read-only endpoint security audit and **unauthorized-use investigation** tool (single self-contained `.exe`, no installer, no internet required)

What's new in 3.0: IMI Guard can now answer the question a field audit team is actually sent to answer: **"did someone break the rules on this PC?"**

- **Organization policy pack (`policy.json`)** — declare what the site permits: internet/hotspot/tethering allowed or not, approved Wi-Fi networks, approved USB devices (by serial), and working hours. **Without a policy, findings are neutral observations; with one, the same evidence becomes a confirmed policy violation.** The tool never guesses what your site permits.
- **Mobile hotspot & tethering detection (`WG-UAU-001/002`)** — catches a phone used to bypass a no-internet rule, via saved SSIDs, Windows' per-network first/last-connected dates, and RNDIS/PAN tethering adapters.
- **Removable-media investigation (`WG-UAU-005/006`)** — which USB device (**by serial**), when it was connected, and which files on it were opened. Includes a **drive-letter-reuse guard**: a file open is only attributed to a USB device if it happened *inside that device's connection window*, because drive letters get reused and innocent internal-disk files must never be reported as USB activity.
- **Connection sharing / hosted network (`WG-UAU-004`)** and **after-hours sign-in review (`WG-UAU-007`)**.
- **Separate Unauthorized-Use verdict** — reported independently of the security-posture score, because a hardened PC can still have been misused and a badly configured PC can be entirely innocent.
- **Case record + evidence manifest (chain of custody)** — `case.json` records who ran the audit, where, why, and under whose authority; `evidence_manifest.json` records the **SHA-256 of every output file**. Verify any package later with `IMIGuard.exe --verify "<folder>"` — it reports **INTACT**, or names every modified/missing/added file.
- **Program-execution history (`WG-EXE-*`)** — recovers *which programs were run and when* from Prefetch, BAM/DAM, and UserAssist, even if the program was since deleted. Flags a **network-restriction bypass tool** being executed (`WG-EXE-002` , e.g. Psiphon/Ultrasure/a personal VPN) and a **program run from a USB stick (`WG-EXE-001`)** — with the same drive-letter-reuse guard as the file-access rule. Also parses the **long-retention artifacts ShimCache and Amcache (`WG-EXE-004/005`)** — which survive when Prefetch/BAM are disabled or rolled over, include programs no longer on disk, and (Amcache) carry each file's **SHA-1 hash** — and flags a bypass tool found there even weeks after it was deleted (`WG-EXE-006`).
- **Data-movement / exfiltration evidence (`WG-EXFIL-*`)** — the modern equivalent of the USB stick: a **personal cloud account** signed in on a work PC (OneDrive/Dropbox/Google Drive/MEGA..., with the account and synced folder), background **BITS** transfers, and **staged archives** ready to move. All read **offline** from local traces — the tool never contacts a cloud service.
- **Pre-flight "Scan Environment & Reliability" panel** — at the very top of every report (and the headless output). It records whether the scan ran **elevated**, the **PowerShell language mode** and **execution policy**, and any **third-party AV/EDR** present, then rates the run **Reliable / Reduced / Unreliable**. Its job is to stop a scan that a locked-down or AV-heavy PC quietly starved of evidence from being mistaken for a clean endpoint.
- **ACL-aware service severity (`WG-SVC-004/005`)** — instead of flagging every service under a user profile as High, the tool now checks the real question: **can a standard user overwrite a binary**

that a privileged service runs? If yes → **Critical** (a working local privilege escalation); if the binary is not user-writable → **not flagged at all**. This removes the "AppData = High" false positives.

- **Root-cause correlation** — when one app surfaces as several findings (a service + a COM entry + a process + a signature), they are grouped under one root cause. Only the **primary** is scored; the rest become **supporting observations** that raise confidence without deducting points again, so a single app no longer inflates the score.
- **Investigation timeline** — every dated event (sign-ins, USB connect/remove, file opens, program executions, network joins, Defender detections, log-clearing, service installs) is placed on one chronological view, and **correlated sequences** are called out so an investigator sees "*USB connected → bypass tool run → file opened, all after hours*" as one episode. Exported as `timeline.json / timeline.csv`. The timeline reports its own **reliability** (a cleared log or unverified clock downgrades it).
- **Accurate finding types** — every finding now carries a precise type (Confirmed misconfiguration, Confirmed exposure, Historical security event, Heuristic review, Change/activity review, Malware indicator, Informational context...) instead of labelling everything "Suspicious."
- **Honest baseline status** — a first-run report says "*baseline created this scan; change detection available on the next scan*" instead of implying every finding is new. And the compliance section separates the **primary CIS requirement assessed** from **related** NIST/Essential-8 references (which are adjacent control families, not exact equivalents).

Evidence honesty (read this): Windows does **not** log file copies by default. IMI Guard can prove a specific device *was connected* and *when*, and that files on a removable drive *were opened* and *when*. It **cannot** prove a file was copied, and the absence of these artifacts does **not** prove no data was taken. The reports say so explicitly. The tool establishes *what* and *when* — never *why*. See §29.

What's new in 2.5 (safety release): technician commands now follow the exact rule, not its category or a word in its title. Previously a LAN-exposure finding ("LAN host exposes sensitive **service**") and the usbipd finding could inherit the service-removal template — including `sc.exe delete` — and a finding that merely reported Defender's *detection history* could be handed Defender *exclusion-removal* commands. Routing is now an explicit rule-id → template map with a **destructive-command guard**: review-only findings and all Informational findings can never emit a delete/disable/remove step, and each finding records its `remediation_template + allow_destructive` in `findings.json / findings.csv` so the routing is auditable. **ATT&CK mappings corrected**: a scheduled task is T1053.005 (not T1547.001); print monitors are T1547.010 (own rule WG-PER-025); and the mappings that asserted adversary behaviour without evidence were **removed** — USB history is no longer T1091, a VPN adapter is no longer T1572, and a user-writable path alone is no longer T1036. The report section is now **"MITRE ATT&CK References"** (not "Coverage"), because these are associations, not confirmed technique detections.

What's new in 2.4: a hash-reputation module that convicts files by their SHA-256. Every hashed file (autostart, service, listener, suspicious process) is checked against a **known-bad denylist** — a built-in seed (the harmless EICAR test hash) plus your own `hash_denylist.json` from incident-response/threat-intel feeds. A match raises **WG-HASH-001 (Critical, Confirmed/High confidence)** — a deterministic identity match, not a heuristic. Optionally, and **off by default**, a consent-gated **VirusTotal lookup** (`hash_reputation.json`) checks unrecognized files online, sending **only the SHA-256** — never file content, paths, or machine data — and raises **WG-HASH-002 (High)**. With no config and no API key, the tool stays fully offline and never makes a network call. See §8.5 and §28.1.

What's new in 2.3: signature findings now **separate genuinely unsigned from unresolved** (an evidence gap — access denied, unsupported file, not checked — is never labeled "unsigned"); **Defender events show what they were** (event type + threat name, not just a count) with severity by type (remediation failure = High, detected-and-removed = Medium); **service-install findings list the actual service names and paths** with severity by location; a **type-name evidence guard** guarantees no

System.Collections... artifact ever reaches a report; and the malware dimension is renamed **Threat Indicator Risk**. This release is about trustworthy, evidence-backed findings — fewer, better, explainable.

What's new in 2.2: a **CIS-aligned compliance engine** (~40 Windows controls with Pass/Fail, a compliance score, and CIS/NIST/Essential-8 references — §26); an **advanced security-policy / hardening** collector (WDigest, LSA protection, Credential Guard, UAC, NLA, ASR, audit policy, account & lockout policy — §27); **process-lineage detection with signature + SHA256** (Office/browser → script chains, unsigned processes from user-writable paths — §28); **browser-extension permission scoring**; a **trusted-publisher allowlist** that reduces false positives on signed vendor binaries; and **evidence-quality** (Complete / Partial) on every finding.

From 2.1 (“Evidence & Confidence”): per-finding **confidence** and **type**, confidence-aware **scoring**, **Audit Completeness**, **Risk by Area**, and three intelligence modules — vulnerable/end-of-life software, a Windows Event ID reference, and LAN device intelligence. See §10–§12 and §23–§28.

Table of contents

1. Overview
2. Requirements
3. Quick start (graphical)
4. Audit modes
5. Evidence modules
6. Command-line (headless) usage
7. Fleet deployment recipes
8. Configuration files
9. Output files
10. Reading the report
11. Severity and scoring model
12. Finding structure and rule families
13. Network connection history module
14. USB device history module
15. Active LAN scan (opt-in)
16. Baseline comparison
17. Whitelisting workflow
18. Remediation and the action guide
19. Safety, ethics, and privacy
20. Building from source and code signing
21. Running the tests
22. Troubleshooting
23. Vulnerable / end-of-life software module
24. Windows Event ID reference
25. LAN device intelligence
26. CIS-aligned compliance engine
27. Advanced security policy & hardening
28. Process lineage & signatures
29. Unauthorized-use & data-movement investigation

30. Field operator playbook

- [Appendix A — rule ID families](#)

1. Overview

IMI Guard collects Windows endpoint security configuration and activity **safely and read-only**, applies a detection rule engine, and produces professional HTML / PDF / JSON / CSV reports with severity ratings, a security score, MITRE ATT&CK mapping, and technician remediation guidance.

It answers questions such as: Is Defender healthy? Is RDP exposed? What remote-access tools are installed? Which networks, VPNs, and USB devices has this machine used, and when? What listening services are exposed? What changed since the last scan?

It runs two ways:

- **Graphical app** — for a technician sitting at (or remoted into) the machine.
- **Headless CLI** — for unattended, fleet-wide use via RMM, SCCM, Intune, GPO, or a scheduled task, with a risk-based exit code.

The tool never modifies system settings, never deletes files, never runs remediation, and never uploads data.

2. Requirements

- Windows 10/11 or Windows Server 2016+.
- **Administrator rights.** The executable requests elevation automatically (embedded manifest). Security, Defender, Firewall, RDP, SMB, BitLocker, event-log, and network-history evidence require it. Run unelevated and those collectors report "Limited".
- Windows PowerShell 5.1 (present on all supported systems) — used internally by the collectors.
- No installation, .NET download, or package manager is required; the tool uses the .NET Framework runtime already present on Windows.

3. Quick start (graphical)

1. Double-click **IMIGuard.exe** and approve the Administrator (UAC) prompt.
2. Choose an **Audit mode**: Quick, Full, or Incident.
3. Tick the optional **Evidence modules** you want (see §5). The **Active LAN scan** is off by default and asks for confirmation.
4. Click **Start Audit**. A progress bar and live run log show each collector. Click **Cancel** to stop.
5. When it finishes, use **Open Report**, **Open PDF**, or **Open Output Folder**.

Each run writes to a new timestamped folder under `audit_results\` next to the executable (or under `--out`).

4. Audit modes

Mode	Target runtime	Purpose
Quick	~1-2 min	Fast workstation health check; first-level triage.
Full	~5-10 min	Complete endpoint review; client report; periodic audit.
Incident	~5-15 min	Suspected compromise; widens event-log and history look-back windows and file-recency windows.

Mode affects how far back event-based collectors look (for example RDP/logon/PowerShell/Defender event windows and network/USB history depth), and which modules are included by default. Quick includes the core collectors plus anything you explicitly tick; Full and Incident include browser and USB history automatically.

5. Evidence modules

Optional collectors, selectable in the GUI or via CLI flags:

Module	GUI default	CLI flag to disable/enable
Established connections	on	<code>--no-connections</code>
Developer / VM environment (WSL, Docker, VMs)	on	<code>--no-developer</code>
Browser extension inventory	on	<code>--no-browser</code>
USB / device history	on	<code>--no-usb</code>
Signature / hash audit	on	<code>--no-reputation</code>
Baseline comparison	on	<code>--no-baseline</code>
Passive LAN inventory	off	<code>--lan</code> to enable
Active LAN scan (aggressive)	off	<code>--active-scan</code> to enable

The **Network connection history** and core security collectors always run.

6. Command-line (headless) usage

Any command-line switch puts the tool in headless mode (no window). Run from an elevated context (fleet tools run as SYSTEM, which satisfies this without a prompt).

```
IMIGuard.exe --silent --mode Full --out D:\audits
```

Options

Flag	Meaning
<code>--mode <Quick Full Incident></code>	Audit depth (default: Full).
<code>--out <folder></code>	Base output folder (default: <code>audit_results</code> next to the exe).
<code>--config <file></code>	

Flag	Meaning
	Path to <code>winguard.config.json</code> for branding/engagement.
<code>--baseline / --no-baseline</code>	Compare against the previous baseline (default: on).
<code>--lan / --no-lan</code>	Passive LAN inventory (default: off).
<code>--active-scan</code>	Opt-in aggressive scan of the local subnet (default: off).
<code>--no-connections</code> <code>--no-developer</code> <code>--no-browser</code> <code>--no-usb</code> <code>--no-reputation</code>	Skip the named module.
<code>-h, --help</code>	Show usage.

Exit codes (so automation can branch on risk):

Code	Meaning
0	Excellent (score $\geq$ 90)
1	Good (75–89)
2	Needs improvement (60–74)
3	High risk (40–59)
4	Critical risk ($<$ 40)
64	Usage error (bad arguments)
65	Runtime error (audit failed)
66	Cancelled

The headless run prints progress lines and a final summary (score, band, severity counts, output folder, report path) to the console.

7. Fleet deployment recipes

All examples assume the tool runs elevated (as SYSTEM or an admin account).

Scheduled task (run as SYSTEM), collect to a share:

```
schtasks /Create /TN "IMI Guard" /RU SYSTEM /SC WEEKLY /D SUN /ST 02:00 ^
/TR "\"C:\Tools\IMIGuard.exe\" --silent --mode Full --out \\fileserver\audits"
```

RMM / SCCM / Intune script step — push the exe (and optionally `winguard.config.json`) to a folder, then run:

```
IMIGuard.exe --silent --mode Full --out C:\ProgramData\WinGuard --config C:
\ProgramData\WinGuard\winguard.config.json
```

Branch on the exit code: for example treat `>= 3` (High/Critical risk) as an alert. Collect the timestamped output folder (or just `audit_metadata.json` + `findings.json`) back to a central location; the **asset id** in each report links repeat scans of the same machine.

GPO startup script — same command line in a startup PowerShell/batch script (startup scripts run as SYSTEM).

8. Configuration files

All three live next to the executable and are created with safe defaults on first run. Edit them with any text editor (UTF-8 JSON).

8.1 winguard.config.json — branding / engagement

Populates the report cover, PDF, and `audit_metadata.json`. All fields optional.

```
{
  "OrganizationName": "NLC Cyber",
  "ClientName": "Acme Corp",
  "EngagementId": "ENG-2026-014",
  "AnalystName": "R. Bell",
  "Classification": "CONFIDENTIAL",
  "LogoPath": "C:\\brand\\logo.png",
  "ReportFooter": "Prepared by NLC Cyber under SOW ENG-2026-014."
}
```

- `Classification` renders as a banner at the top and bottom of the HTML report and on the PDF cover.
- `LogoPath` is embedded into the HTML report as a data URI (PNG/JPG/GIF/SVG, up to ~500 KB), keeping the report a single portable file.

8.2 whitelist.json — suppress approved findings

Enabled by default in 2.1, but ships with **no active suppressions**, so nothing is hidden until you add an entry. Add match entries to approve expected findings on your fleet. Suppressed findings are removed from the score and report but preserved in `whitelist_suppressed_findings.json`.

```
{
  "Enabled": true,
  "Ids": ["WG-REMOTE-001"],
  "TitleContains": [],
  "EvidenceContains": ["Tailscale"],
  "Entries": [
    { "Enabled": true, "Id": "WG-REMOTE-001", "Category": "Remote Access",
      "EvidenceContains": "Tailscale", "Reason": "Approved remote-access tool" }
  ]
}
```

Match by exact rule `Ids`, by `TitleContains` / `EvidenceContains` substrings, or by structured `Entries` (id + category + title/evidence substrings, all optional).

8.3 rulecatalog.json — MITRE mapping and severity tuning

Enriches findings by rule id with a MITRE ATT&CK `Technique`, `References` URL, and an optional `SeverityOverride` — all **without recompiling**. Built-in defaults ship in code; the file (written on first run) overrides them.


```
{
  "Enabled": true,
  "Rules": {
    "WG-RDP-001": { "Technique": "T1021.001",
                    "References": "https://attack.mitre.org/techniques/T1021/001/",
                    "SeverityOverride": "" },
    "WG-NET-005": { "Technique": "", "References": "", "SeverityOverride": "Low" }
  }
}
```

Keys may be exact ids (`WG-RDP-001`) or a family prefix ending in `-` (`WG-RDP-`). A valid `SeverityOverride` (`Critical` / `High` / `Medium` / `Low` / `Info`) retunes that rule; leave empty to keep the engine default.

8.4 `publisher_allowlist.json` — trusted software publishers

Extends the built-in trusted-publisher list used to suppress false positives on validly-signed vendor binaries (see §28). Add your own approved publisher names (matched against the Authenticode signer subject).

```
{ "Enabled": true, "Publishers": ["Contoso Ltd", "Fabrikam Inc"] }
```

8.5 `hash_denylist.json` — known-bad file hashes (offline)

Files whose SHA-256 appears here are convicted as **WG-HASH-001 (Critical)**. Drop in hashes from your incident-response cases or threat-intel feeds. The tool ships with one built-in entry — the harmless **EICAR** test hash — so you can verify the module works. Both files below are auto-created next to the executable on first run.

```
{ "Enabled": true, "KnownBad": [
  { "Sha256": "aa..64hex..ff", "Severity": "Critical", "Source": "IR-2026-014", "Note":
    "Cobalt Strike loader seen in case 014" }
] }
```

8.6 `hash_reputation.json` — optional online VirusTotal lookup (off by default)

Online lookup is **disabled** until you set `OnlineEnabled` to `true` and supply a VirusTotal API key. When enabled, only files not already trusted by a recognized publisher are queried, **only the SHA-256 is sent**, and the number of lookups per run is capped by `MaxOnlineLookups`. A match raises **WG-HASH-002 (High)**.

```
{ "Enabled": true, "OnlineEnabled": false, "VirusTotalApiKey": "", "MaxOnlineLookups": 20 }
```

Privacy: a SHA-256 is a one-way fingerprint, not the file. No file content, path, or machine data ever leaves the host. Leave `OnlineEnabled` false to keep the audit 100% offline.

9. Output files

Per timestamped folder under `audit_results\<COMPUTER>_<yyyyMMdd_HHmss>\`:

File	Purpose
<code>report.html</code>	Interactive report (donut/bar charts, search, filters, sortable table, collapsible sections). Print-ready.
<code>report.pdf</code>	Branded PDF summary.
<code>audit_metadata.json</code>	Aggregation anchor: schema version, tool version, asset id , score, band, counts, engagement, collector health , risk by area .
<code>winguard_run.log</code>	Tool-execution audit trail: options, per-collector timing, errors, collector-health summary.
<code>findings.json</code> / <code>findings.csv</code>	Machine-readable findings (consolidated, with count, confidence , type , status , evidence quality , and MITRE technique).
<code>compliance.json</code> / <code>compliance.csv</code>	CIS-aligned control results (id, section, Pass/Fail, frameworks, evidence, remediation).
<code>technician_action_guide.md</code>	Per-finding check / remediation / validation commands.
<code>summary.txt</code>	Plain-text executive summary.
<code>baseline_current.json</code> / <code>baseline_diff.html</code>	Baseline snapshot and change diff.
<code>whitelist_suppressed_findings.json</code>	What the whitelist suppressed.
<code>raw_evidence/*.json</code>	Raw per-module evidence (redacted).
CSV exports	<code>network_ports</code> , <code>installed_apps</code> , <code>services</code> , <code>scheduled_tasks</code> , <code>file_reputation</code> , <code>rdp_history</code> , <code>logon_history</code> , <code>network_history</code> , <code>usb_history</code> , <code>active_lan_hosts</code> .

Asset id — a stable, non-reversible identifier (`WGA-...`) derived from the machine identity, so repeat scans of the same endpoint can be tracked and aggregated over time.

10. Reading the report

`report.html` opens in any browser, fully offline.

- **Cover** — branding, classification banner, computer, mode, asset id, generation time.
- **Dashboard** — security score (colored by band), a severity donut, and a top-categories bar chart.
- **Executive summary** — score narrative plus system identity, including **evidence completeness** and **score confidence** (see §11).
- **Audit Completeness** — how many collectors completed vs. returned limited evidence, the resulting **score confidence** (High/Medium/Low), and whether the tool ran with Administrator rights. Limited collectors lower *confidence*, not the score.
- **Risk by Area** — a manager-facing rollup of the risk band for each domain: Remote Access, Persistence, Malware, Patch/Hardening, Developer, Network.
- **Summaries** — a plain-English **For Management** list of the top risks, and a **For Technicians** quick-reference (listening ports, suspicious processes, recent RDP sessions, remote-access indicators, persistence findings).
- **Findings table** — every finding, one row each, now with a **Confidence** column (High/Medium/Low) and a **type** badge (Confirmed / Suspicious / Informational). Use the **search box** (matches title, evidence, category, confidence, technique), the **severity filter chips** (All / Critical / High / Medium / Low / Info), and

click **column headers to sort** (severity or confidence). A finding seen many times shows an instance count badge (e.g. **x68**).

- **Correlated Risk** (**WG-CORR-**) — dangerous *combinations* elevated into a single high-signal finding (e.g. network-exposed RDP **plus** weak authentication → Critical; antivirus disabled **plus** prior malware detections → Critical). Either condition alone is far less dangerous than the pair.
- **Collapsible sections** — MITRE ATT&CK coverage, findings by category, priority action plan, technician command library, network connection history, USB device history, **Scheduled Tasks** (full inventory, hidden first), active LAN scan (now with **NIC vendor + device type**, see §25), **Windows Event Reference** (see §24), advanced persistence (ASEP), evidence coverage, raw-evidence appendix.

When printed (Ctrl+P), controls are hidden and all sections expand automatically.

11. Severity, confidence, and scoring model

Severity = how bad it is if true: Critical, High, Medium, Low, Info. **Confidence** = how reliable the detection is, *independent* of severity: High, Medium, Low. **Type** = the nature of the finding:

Type	Meaning
Confirmed	A deterministic, config-readable fact (e.g. Defender is off, an account has no password).
Suspicious	An indicator that warrants review but may be legitimate (e.g. an unrecognized LSA package).
Informational	Context, not a problem (inventory, history) — never affects the score.

Confidence-aware score with diminishing returns (2.1). Each finding is assigned a base weight from **both its severity and confidence**:

Severity	Confidence	Base weight
Critical	Confirmed / High	35
Critical	otherwise	25
High	High	15
High	Medium	8
High	Low	3
Medium	any	5
Low	any	1
Info / Informational	—	0

Those weights are then sorted largest-first and **damped by a decay** before being subtracted from 100, so the single worst issue counts in full but a long tail of lower findings can't pile up and floor the score. In practice, **one Critical plus a tail of High/Medium findings lands mid-range (≈ High risk)**, while a genuinely compromised endpoint (several confirmed Criticals) still scores in the Critical band. Informational findings never move the score. Minimum is 0. Repeated instances of the same rule are **consolidated into a single finding** (with an instance count) before scoring, so forty services with unquoted paths cost one weight, not forty.

Score bands:

Score	Band
90-100	Excellent
75-89	Good
60-74	Needs improvement
40-59	High risk
0-39	Critical risk

Score confidence (audit completeness). Separately from the score, the report states how much of the planned evidence was actually collected. If collectors were limited (e.g. run without Administrator, or a module timed out), the **score confidence** drops to Medium or Low. A limited collector is an *evidence gap*, not a clean result — it never silently improves the score.

12. Finding structure and rule families

Every finding carries: `Id`, `Severity`, `Confidence`, `FindingType`, `Status`, `Category`, `Title`, `Evidence`, `Risk`, `Recommendation`, `FalsePositiveNotes`, `Count`, `Technique` (MITRE), and `References`. The new fields are also written to `findings.csv` and `findings.json`.

- **Status** is a triage lifecycle field (default `New`) for teams that run the tool repeatedly — you can mark a finding `Confirmed`, `False Positive`, `Approved Exception`, `Remediated`, or `Accepted Risk` in your own tracking.

Rule ids are grouped into families (see [Appendix A](#)).

13. Network connection history module

Answers "which networks, VPNs, and modems has this machine used, and when." Data sources and realistic retention:

- **Known networks (registry)** — every network the machine has joined: name, type (wired / wireless / VPN / mobile broadband), category (public / private / domain), **first-connected** and **last-connected** dates, and **gateway MAC** (identifies the physical router/modem). Persists for **months to years**. Requires Administrator.
- **Connect / disconnect timeline** — NetworkProfile and WLAN-AutoConfig events (Wi-Fi SSIDs, times). Retention is **days to weeks** (event-log size dependent).
- **VPN / dial-up** — RasClient events (connection name + server + times), configured VPN profiles, and third-party VPN adapters (WireGuard, OpenVPN, Tailscale, ZeroTier, AnyConnect, GlobalProtect, etc.).
- **Cellular / modem / tethering** — adapters by media type (WWAN, mobile broadband, RNDIS phone tethering).

So *which* networks/VPNs = months of history; the *minute-by-minute* timeline = however long the logs hold. Findings: `WG-NETH-001` (history collected), `WG-NETH-002` (VPN usage), `WG-NETH-003` (cellular/modem/tethering), `WG-NETH-004` (public networks). CSV: `network_history.csv`.

14. USB device history module

Per USB device (present and previously connected):

- Friendly name, class, serial, manufacturer.
- **First installed, last connected, last removed** timestamps.
- **Last-session duration** (e.g. `2.7 h (still connected)` , `11.3 h`).
- Present / not-present state.

Windows does not track *cumulative* connection time, so the tool reports the last session plus first/last dates — the maximum reliably available. Findings: `WG-USB-001` (storage history), `WG-USB-002` (usbipd USB/IP service running). CSV: `usb_history.csv`.

15. Active LAN scan (opt-in)

An **aggressive, opt-in** discovery scan of the **directly-connected local subnet only**. Off by default; the GUI asks for confirmation and the CLI requires `--active-scan`.

What it does: a concurrent ICMP sweep of the local /24, then a TCP **connection** probe of reachable hosts on common ports (21, 22, 23, 53, 80, 139, 443, 445, 3306, 3389, 5432, 5900, 6379, 8080, 8443, 9200), plus reverse-DNS names and the ARP table. It performs **no exploitation and no credential testing**, and never scans beyond the local subnet.

Findings: `WG-LANSCAN-001` (scan summary), `WG-LANSCAN-002` (a neighbor host exposes a sensitive service — RDP/SMB/Telnet/SSH/database/VNC). CSV: `active_lan_hosts.csv`.

Each discovered host is also enriched with its **NIC vendor and device type** — see §25.

Authorization: only run the active scan on networks you own or are authorized to test.

16. Baseline comparison

On each run (unless `--no-baseline`), the tool writes a `baseline_current.json` snapshot and, if a previous one exists, produces `baseline_diff.html` and baseline findings (`WG-BASE-...`) for newly appeared: local admins, listening ports, auto-start services, scheduled tasks, remote-access tools, Defender exclusions, and trusted root certificates. This surfaces change over time — for example a new admin account or a new exposed port since last audit.

17. Whitelisting workflow

1. Run an audit and review findings.
2. For each finding that is expected/approved on this fleet, add a match rule to `whitelist.json` (§8.2) with a `Reason`.
3. Set `"Enabled": true`.
4. Re-run. Whitelisted findings drop out of the score and the main report but are preserved in `whitelist_suppressed_findings.json` for auditability.

Use whitelisting to reduce noise (e.g. an approved remote-support tool) without losing the record.

18. Remediation and the action guide

The tool **never remediates**. For every finding it provides guidance only, in three parts — **Check, Remediate, Validate** — as ready-to-paste PowerShell. This appears in the report's action sections, in `technician_action_guide.md`, and in the GUI **Action Guide** tab when a finding is selected.

Workflow: check the evidence, preserve incident evidence where relevant, get approval, then run remediation manually, and re-run the audit to confirm.

19. Safety, ethics, and privacy

- **Read-only.** No cleanup, service changes, exploitation, or persistence. Remediation commands are guidance only and are never executed by the tool.
- **No secret collection.** No passwords, cookies, saved credentials, tokens, private keys, browser history, or personal documents. Output is passed through a redaction filter for accidental secrets (password/token/apikey/Authorization: Bearer/private_key patterns).
- **No upload.** Nothing leaves the machine; no internet connection is required.
- **Active scanning is opt-in**, local-subnet only, and discovery/probe only.
- **Authorized use only** — your own systems, or systems you have documented permission to audit.

20. Building from source and code signing

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -File .\build.ps1
```

Uses the Windows .NET Framework C# compiler already on the system (no package install). The build embeds the application manifest (Administrator + DPI aware) and produces `IMIGuard.exe`, a `release\` folder, and `IMIGuard_release.zip`.

Code signing (recommended for enterprise deployment so SmartScreen/EDR trust the binary) runs automatically if a certificate is provided via environment variables *before* building:

- `WG_SIGN_THUMBPRINT` — thumbprint of a code-signing cert in the local certificate store, **or**
- `WG_SIGN_PFX` — path to a `.pfx` file (plus optional `WG_SIGN_PFX_PASSWORD`)
- `WG_SIGN_TIMESTAMP` — optional RFC3161 timestamp URL (default DigiCert)

Without one, the build succeeds but the executable is unsigned.

21. Running the tests

Rule-engine regression tests feed synthetic evidence through the detection + consolidation pipeline and assert the expected findings:

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -File .\tests\run_tests.ps1
```

Exit code 0 means all tests passed. Add fixtures in `tests\Tests.cs` when you add or change rules.

22. Troubleshooting

Symptom	Cause / fix
UAC prompt / won't start without elevation	Expected — the tool requires Administrator. Fleet tools run as SYSTEM and satisfy this silently.
Many collectors show "Limited"	Running unelevated. Security/Defender/Firewall/RDP/SMB/BitLocker and network-history registry data need Administrator.
Network "known networks" empty	Requires Administrator (the <code>NetworkList</code> registry is admin-only). The event-based timeline still works unelevated.
SmartScreen / EDR warning on the exe	The binary is unsigned unless you built with a signing cert (\$20). Sign it for trusted deployment.
Scan feels slow	Full/Incident do a lot; collectors run in parallel. The <code>winguard_run.log</code> shows per-collector timing if you need to find a slow one.
Headless run shows no console output	It attaches to the parent console when launched from one; when launched detached (scheduled task), rely on the report files and <code>winguard_run.log</code> .
Score is 0	Enough Critical/High findings to floor it. Review Critical findings first; use the whitelist for approved items.

23. Vulnerable / end-of-life software module

Matches the installed-software inventory against a **built-in, offline catalog** of high-signal cases and raises `WG-CVE-001` findings under the **Vulnerability** category.

- **Actively-exploited, version-bounded** — flagged only when the installed version is at or below the affected ceiling, for example **WinRAR before 6.23** (CVE-2023-38831, actively exploited), **7-Zip before 21.07** (CVE-2022-29072), older VLC, Zoom, and Notepad++.
- **End-of-life runtimes / plugins** — flagged whenever present: Adobe Flash, Shockwave, Silverlight, QuickTime for Windows, Python 2, Internet Explorer, end-of-life OpenSSL/Node.js.

This is a curated seed list, **not** a full CVE feed, and it does not need internet access. Because a registry version string is not always precise, these findings are **Suspicious** by nature — verify the installed version before acting. The finding text names the CVE (where applicable) and the recommended fix.

24. Windows Event ID reference

The **Windows Event Reference** section of the report explains, in plain English, only the security-relevant Windows Event IDs that **actually appeared** in this audit's evidence — logon (4624/4625), account and group changes (4720/4732/...), service installation (7045/4697), scheduled-task creation (4698), log clearing (1102), RDP/Terminal-Services (1149/21/24/25), Defender detection/action (1116/1117/1118), and PowerShell script-block logging (4104). Each row shows the ID, its name, how many times it was seen, and what it means, so the logon/RDP/service tables elsewhere in the report are readable without a separate lookup. Fully offline.

25. LAN device intelligence

When the **active LAN scan** runs, each discovered host is enriched — fully offline — with:

- **NIC vendor** — resolved from the host's MAC address (captured in the ARP table) against a built-in OUI vendor table covering common networking, PC, mobile, camera, printer, and IoT vendors. An unrecognized vendor is shown as **Unknown**, which is itself useful signal for spotting an unexpected device.
- **Device type** — a best-effort classification from the vendor plus open ports and gateway status: *Router / access point, Printer, IP camera / NVR, IoT / embedded device, Virtual machine, Windows host, Linux/Unix host, Apple device, Mobile device, or Unknown*.

These appear as **Vendor (MAC)** and **Device type** columns in the Active LAN Scan table, so a rogue or unexpected device on the subnet stands out at a glance. No extra collection is performed — the intelligence is derived from data the scan already gathered.

26. CIS-aligned compliance engine

Alongside the risk findings, IMI Guard evaluates a **curated set of ~40 high-value Windows security controls** against the collected evidence and reports each as **Pass / Fail / Not Assessed**, producing a separate **compliance score** (percentage of *assessed* controls that pass).

Controls are grouped by area and cross-referenced to public frameworks:

Area	Example controls
Account & Lockout Policy	min password length ≥ 14 , complexity, history, lockout threshold, cleartext storage off
Local Accounts	Guest disabled, blank-password use limited
Credential Protection	WDigest cleartext off, LSA protection (RunAsPPL), NoLMHash, NTLMv2, anonymous SAM restricted, Credential Guard
User Account Control	UAC enabled, secure-desktop prompt, remote-UAC token filtering
Microsoft Defender	real-time protection, behavior monitoring, PUA protection, tamper protection, ASR rules
Windows Firewall	all profiles enabled, inbound default block
Network Security	SMBv1 disabled, SMB signing, AutoRun off, AlwaysInstallElevated off
Remote Access	RDP Network Level Authentication
Audit Policy	logon, process creation, credential validation, account management, policy change
Logging & Platform	PowerShell script-block logging, BitLocker, Secure Boot, Windows Update, LAPS

Honesty by design: this is a *CIS-aligned* control set, **not** full certified CIS-CAT parity. A control the tool cannot measure from the collected evidence (for example, because a collector was limited or the machine lacks that feature) is marked **Not Assessed** and **excluded from the score** — it never counts as a silent pass or fail. Each result carries the control id, section, status, severity (for failures), framework references, evidence, and remediation.

Outputs: a **CIS-Aligned Compliance** section in the HTML report (score cards + a full controls table, failed controls first), a compliance summary in the PDF, and `compliance.json` / `compliance.csv` for aggregation. The compliance score is also in `audit_metadata.json` under `compliance_cis`.

27. Advanced security policy & hardening

A dedicated collector gathers deeper hardening evidence used by both the findings engine and the compliance engine, all read-only:

- **Account & lockout policy** via `secedit` (minimum length, complexity, history, max age, lockout threshold/duration) — locale-independent.
- **Security options** (registry): WDigest cleartext caching, LSA protection (RunAsPPL), NoLMHash / LmCompatibilityLevel, anonymous-SAM restriction, UAC settings, RDP NLA, AutoRun, PowerShell logging, `AlwaysInstallElevated`.
- **Credential Guard** / **VBS** status, **Defender** PUA/tamper/controlled-folder-access, **Attack Surface Reduction** rule enforcement count, **LAPS** presence.
- **Audit policy** per subcategory (keyed by stable GUID, locale-independent).

The most dangerous misconfigurations are also raised as prioritized findings: `WG-HARD-010` (WDigest stores cleartext credentials), `WG-HARD-011` (AlwaysInstallElevated privilege escalation), `WG-HARD-013` (RDP without NLA), `WG-HARD-014` (script-block logging off), and `WG-HARD-015` (weak password/lockout policy). The full checklist lives in the compliance section (§26).

28. Process lineage & signatures

A focused process collector captures **parent-child process relationships** and, for the processes that matter, their **Authenticode signature and SHA256 hash**. It flags two patterns:

- **Suspicious process chains** (`WG-PROC-001`) — an Office application or browser spawning a script interpreter (`powershell`, `cmd`, `wscript`, `cscript`, `mshta`, `rundll32`, `regsvr32`), a classic living-off-the-land execution technique. The lineage itself is the signal, regardless of the child's signature.
- **Processes from user-writable paths** (`WG-PROC-002`) — executables running from `Users / AppData / Temp / Downloads / ProgramData`, with severity raised when the binary is **unsigned or from an unrecognized publisher**.

The **trusted-publisher allowlist** (built-in vendors such as Microsoft, Google, NVIDIA, Intel, Dell, JetBrains, Docker, plus anything you add to `publisher_allowlist.json`) suppresses processes that are validly signed by a known vendor, keeping the noise down. Each finding includes the signature status, signer, and a short SHA256 for reputation lookups.

28.1 Hash reputation (SHA-256 conviction)

Where §28 asks "is this file *signed*?", the hash-reputation module asks "is this file *known bad*?". After the signature/hash collector runs, every SHA-256 it produced — across startup items, services, listeners, and suspicious processes — is checked, de-duplicated by hash:

- **Offline denylist (always on)** — the hash is compared against a **known-bad list**: a built-in seed (the harmless **EICAR** anti-malware test hash) plus your own entries in `hash_denylist.json` (§8.5). A match raises `WG-HASH-001` (**Critical, Confirmed / High confidence**). Because a SHA-256 is a unique fingerprint, this is a deterministic identity match — not a heuristic — and is treated as a confirmed indicator of compromise (isolate the host, preserve the file, follow IR procedure).
- **Online VirusTotal lookup (opt-in, off by default)** — if you enable it in `hash_reputation.json` (§8.6), files **not** already trusted by a recognized publisher are looked up live. **Only the SHA-256 is sent** — never

file content, path, or machine data. If anti-malware engines flag it, the tool raises **WG-HASH-002 (High)** with the detection count.

To verify the module end-to-end, place the EICAR test file ([eicar.com](https://www.eicar.com)) in a startup/autorun location, re-run the audit, and confirm **WG-HASH-001** fires. EICAR is harmless and is the industry-standard way to test detection plumbing without real malware.

This is what moves the tool from "shows signs of exposure" toward "identifies a known-bad file" — while staying offline-first: with no config and no API key, no network call is ever made.

29. Unauthorized-use investigation (v3.0)

This is the mode for a field audit team sent to a site to check whether a PC was misused — e.g. *"internet is not allowed here, but we think staff are tethering their phones"* or *"USB is banned, but we think someone copied data."*

29.1 The policy gate — the most important concept

IMI Guard **does not know what your site permits**, so by default it reports unauthorized-use evidence as neutral **observations** (Low / "Informational context"). Supply **policy.json** with **"Enabled": true** and the same evidence is re-classified as a **Policy Violation** with a severity to match.

Policy setting	Effect
<code>InternetAllowed: false</code>	Any non-corporate network becomes a violation
<code>MobileHotspotAllowed: false</code>	Phone-hotspot use becomes High / Policy Violation
<code>UsbTetheringAllowed: false</code>	An RNDIS/PAN tethering adapter becomes High
<code>UsbStorageAllowed: false</code>	Any USB storage device becomes High / Policy Violation
<code>ApprovedNetworks: [...]</code>	Any other SSID is flagged
<code>ApprovedUsbDevices: [...]</code>	Any device whose serial is not listed is flagged
<code>WorkingHours: {...}</code>	Sign-ins outside these hours are raised for authorization review

29.2 What it detects

- **WG-UAU-001 Mobile hotspot / mobile-broadband network used** — from saved Wi-Fi profiles and Windows' `NetworkList` registry, which records the **first and last connection date of every network this PC ever joined**. That fixes *when* it was used.
- **WG-UAU-002 Phone tethering adapter present** — a "Remote NDIS" or "Personal Area Network" adapter only appears once a phone has been connected and shared its data connection.
- **WG-UAU-003 Network used that is not on the approved list.**
- **WG-UAU-004 This PC is sharing its connection outward** (Internet Connection Sharing / Wi-Fi hosted network).
- **WG-UAU-005 USB storage device used** — identified **by serial number**, with first/last connection times.
- **WG-UAU-006 Files on a removable drive were opened while a USB device was connected.**
- **WG-UAU-007 Sign-in activity outside approved working hours** — raised as *authorization review*, never as an accusation.

- **WG-UAU-008 Software prohibited by policy is present** — matches the `ProhibitedSoftware` list in `policy.json` against installed software (and corroborates with execution evidence when the program was actually run). Policy-gated: silent unless the list is filled in.

29.3 The drive-letter-reuse guard (why you can trust WG-UAU-006)

Drive letters are **reused**. An internal disk may hold `D:` for years; a USB stick then mounts as `D:` once, and Windows' `MountedDevices` now maps `D:` → USBSTOR. A naive tool would report *years of the user's own internal-disk files* as "removable media evidence" — and get an innocent person disciplined.

IMI Guard therefore counts a file open as removable-media evidence **only if both** hold:

1. the drive letter is **not** currently a fixed internal disk, **and**
2. the file was opened **inside a window when a USB storage device was actually connected** (derived from the device's recorded arrival/removal times).

File opens that cannot be tied to a device session are **excluded and explicitly disclosed** in the finding ("*a further N file open(s) ... could NOT be tied to a USB connection window and are deliberately NOT counted*"), so the reader can see what was withheld and why.

29.4 What this evidence can and cannot prove

Provable	Not provable
A specific device (by serial) was connected , and when	That any file was copied to or from it
Files on a removable drive were opened , and when	That no data was taken (absence ≠ innocence)
Which networks were joined, and when	Why any of it happened — intent is the investigator's call

Windows does not log file copies unless **object-access auditing** (Audit File System) or DLP is enabled. To *prove* copying on future audits, enable those first. IMI Guard reports only what Windows actually recorded.

29.5 Case record and evidence integrity

Unauthorized-use findings end up in HR and disciplinary processes, where the first question is "*who produced this, when, and under whose authority?*"

- `case.json` — Case ID, organization, site, auditor name, audit reason, approving authority, witness. Printed on the report cover.
- `evidence_manifest.json` — the **SHA-256 of every file** in the output package, plus the case record, the tool's own hash, and the audit's start/end times and time zone.
- **Verify later, on any machine:**

```
IMIGuard.exe --verify "D:\audits\DESKTOP-001_20260713_093012"
```

It prints `RESULT: INTACT` when every file matches its recorded hash, or `RESULT: FAILED` naming each **MODIFIED, MISSING, or ADDED** file. Store the manifest somewhere write-once if the findings may be disputed.

29.6 Data-movement / exfiltration evidence (WG-EXFIL-*)

The USB rules cover the physical stick; these cover the modern equivalent — company data leaving through a **personal cloud account**, a background transfer, or an archive staged ready to move. **Everything here is read entirely offline** from local traces on the disk being audited; the tool never contacts a cloud service.

- **WG-EXFIL-001 Personal / unapproved cloud storage signed in** — OneDrive (Personal vs Business, with the signed-in email and synced folder, from `HKCU\...\OneDrive\Accounts`), Dropbox (`info.json`), Google Drive, MEGA, iCloud, Box, pCloud, Sync.com, Tresorit, Nextcloud. A **personal-looking account on a work PC** (e.g. an `@gmail` / `@outlook` OneDrive) is the giveaway. Maps to **MITRE T1567.002 — Exfiltration to Cloud Storage**.
- **WG-EXFIL-002 Background transfer jobs (BITS)** — upload jobs (and external-URL downloads, as context) with bytes and destination. Maps to **T1197 — BITS Jobs**.
- **WG-EXFIL-003 Staged archives** — large (>20 MB) `.zip/.rar/.7z/...` created recently in Downloads/Desktop/Documents/Temp. Software-distribution downloads (runtimes, SDKs, installers, ISOs) and package-manager caches are **filtered out** so the signal isn't buried in installer noise. Maps to **T1560 — Archive Collected Data**.
- **WG-EXFIL-000 Inventory** — the data-movement channels present, plus per-adapter throughput *since last boot* as context.

These feed the same **Unauthorized-Use & Data-Movement verdict** as the `WG-UAU-*` rules, and staged-archive events (which carry real timestamps) are placed on the **investigation timeline** for correlation with USB, logon, and network activity.

Policy gate (same as unauthorized use): with no policy these are neutral observations. Set `PersonalCloudStorageAllowed=false` in `policy.json` — and list corporate accounts/domains under `ApprovedCloudAccounts` — to turn a personal cloud client into a **confirmed policy violation**.

What it can and cannot prove. A signed-in cloud client proves the sync **path exists** — not that any specific file was uploaded. Adapter byte counters are cumulative **since last boot** and not attributable to any user or app. **Per-application byte accounting (SRUM) is not parsed:** that database is locked on a live system and would require a volume snapshot to read, which is outside this tool's read-only profile. As everywhere else: absence of these artifacts does **not** prove no data left.

30. Field operator playbook

This is the on-site runbook for an IMI investigator. The tool is careful by design, but its output can end up in an HR file or a disciplinary hearing — so **how you run it and how you read it matters as much as the tool itself**. Follow this every time.

30.1 Before you go (pre-engagement)

1. **Get authorization in writing.** Record it in `case.json` (`AuditReason`, `ApprovingAuthority`, `WitnessName`). No authorization → do not run.
2. **Fill in `policy.json`** for the site *before* you arrive: set `InternetAllowed` / `MobileHotspotAllowed` / `UsbStorageAllowed` / `PersonalCloudStorageAllowed`, list `ApprovedNetworks`, `ApprovedUsbDevices`, `ApprovedCloudAccounts`, `ProhibitedSoftware`, and `WorkingHours`. **Set `Enabled: true`**. Without this, every finding is a neutral *observation*, not a policy classification.
3. **Ask the site to allowlist the tool** in their AV/EDR console (by hash or path). This is the single most common cause of a broken scan — `IMIGuard.exe` spawns PowerShell collectors, and Kaspersky/CrowdStrike/etc. will block or slow them. Confirm this is done before you start.

4. **Bring the tool on a clean, write-controlled USB.** Do not ship real `audit_results` on it.

30.2 On the machine

1. **Run as Administrator.** Right-click → *Run as administrator* (GUI), or launch the headless CLI from an elevated shell. Without elevation, Prefetch, BAM, the full Security log, and parts of Defender/Firewall/BitLocker are unavailable.
2. **Read the "Scan Environment & Reliability" panel first — before any finding.**
 - **Reliable (green):** proceed.
 - **Reduced (amber):** usable, but note *why* (not elevated, third-party AV/EDR, limited collectors) and re-run elevated if you can.
 - **Unreliable (red): stop.** Constrained Language Mode or a blocked environment means the collectors returned little — the report is not trustworthy. Get the environment fixed (full-language PowerShell, tool allowlisted) and re-run. **A near-empty "clean" report here proves nothing.**
3. Let the scan finish. It is **read-only** — it changes nothing on the machine.

30.3 Reading results honestly (the non-negotiable part)

1. **What the tool proves vs. what it does not** (this is stated in every relevant finding — do not overstate it):
 - **Proven:** a device (by serial) was *connected* and when; files on a removable drive were *opened* and when; which networks were joined and when; a personal cloud client is *signed in*; a program *ran*.
 - **NOT proven:** that any file was *copied*; that data *was* exfiltrated; **why** any of it happened.
 - **Absence of a finding is not innocence** — Windows does not log file copies by default.
2. **Confirm every unauthorized-use / data-movement finding with the user before it goes in a report.** A policy match can be broader than intended (e.g. `torrent` matching an unrelated product); a drive letter can be reused; a personal cloud account may be sanctioned. Establish *what* and *when*; leave *intent* to the investigation, not the tool.
3. **Correlate, don't cherry-pick.** Use the **investigation timeline** and **root-cause** grouping — a single artifact is weak; a USB connect + file opens + a bypass-tool run in the same window is a story.

30.4 After the scan (chain of custody)

1. The tool writes `evidence_manifest.json` (SHA-256 of every output file + the case record). **Copy the whole output folder to write-once/controlled storage.**
2. **Verify integrity later on any machine:** `IMIGuard.exe --verify "<output folder>"` → prints `RESULT: INTACT`, or names each MODIFIED / MISSING / ADDED file. Run this before the report is relied upon in any proceeding.

30.5 Red lines

- **Never** present a finding as proof of copying, theft, or intent — the tool establishes facts, not motive.
- **Never** act on a **Reduced/Unreliable** scan as if it were complete.
- **Never** run without authorization, and **never** publish or upload a report — it contains the machine's real data (accounts, activity, identifiers).
- **Never** ship the operator's `policy.json` / `case.json` from one site to the next without reviewing it — stale approvals and prohibited lists cause wrong classifications.

Appendix A — rule ID families

Prefix	Area
WG-USR-	Local users / admins / privileges
WG-RDP-	RDP exposure and history
WG-REMOTE-	Remote-access tooling
WG-DEF-	Microsoft Defender health / exclusions / detections
WG-NET-	Network exposure (listening services, databases, dev servers)
WG-NETINT-	Proxy / DNS / hosts / certificates
WG-PER-	Startup and persistence
WG-SVC-	Services (unquoted paths, etc.)
WG-SMB-	SMB shares and configuration
WG-HARD-	Firewall, SMBv1, BitLocker, Secure Boot, UAC, Windows Update
WG-SW-	Installed software risk
WG-CVE-	Vulnerable / end-of-life software (offline catalog)
WG-PATCH-	Patch / update state
WG-EVT-	Security event-log signals
WG-PS-	PowerShell activity
WG-MAL-	Malware indicators
WG-PROC-	Process lineage & signatures
WG-CORR-	Correlated risk (dangerous combinations)
WG-REP-	File signature / reputation
WG-HASH-	Known-bad hash reputation (denylist / VirusTotal)
WG-UAU-	Unauthorized use (hotspot, tethering, USB, after-hours)
WG-EXE-	Program-execution history (bypass tool, run-from-USB)
WG-EXFIL-	Data movement (personal cloud sync, BITS transfers, staged archives)
WG-SVC-004/005	ACL-aware service severity (user-writable service binary)
WG-DEV-	Developer / VM / container exposure
WG-BROWSER-	Browser extensions
WG-LAN-	Passive LAN inventory
WG-LANSCAN-	Active LAN scan results
WG-NETH-	Network connection history (Wi-Fi / VPN / modem)
WG-USB-	USB device history
WG-BASE-	Baseline change detection

Findings are enriched with MITRE ATT&CK techniques via `rulecatalog.json`; see the "MITRE ATT&CK Coverage" section of the report for the techniques observed in a given audit.

Appendix B — Licensing

IMI Guard is licensed with an **offline license file** named `license.key`. The application checks it locally — nothing is sent to any server, and the file is cryptographically signed so it cannot be edited or forged.

Installing your license: place the `license.key` you were given in the **same folder as** `IMIGuard.exe`. That's it. The current license status is shown in the report footer, on the PDF cover, and in the run log.

If no valid license is present, the tool still runs, but reports are marked "UNLICENSED — EVALUATION" and the interface shows a banner. To license the copy, add a valid `license.key`. If your license has expired, contact your supplier for a renewal.

To request or renew a license, contact your IMI Guard supplier.

IMI Guard v3.0 — read-only, defensive, authorized use only. The tool reports what Windows recorded; it does not determine intent.